



Node.js, Express, and MongoDB

NTOU CSE
2025

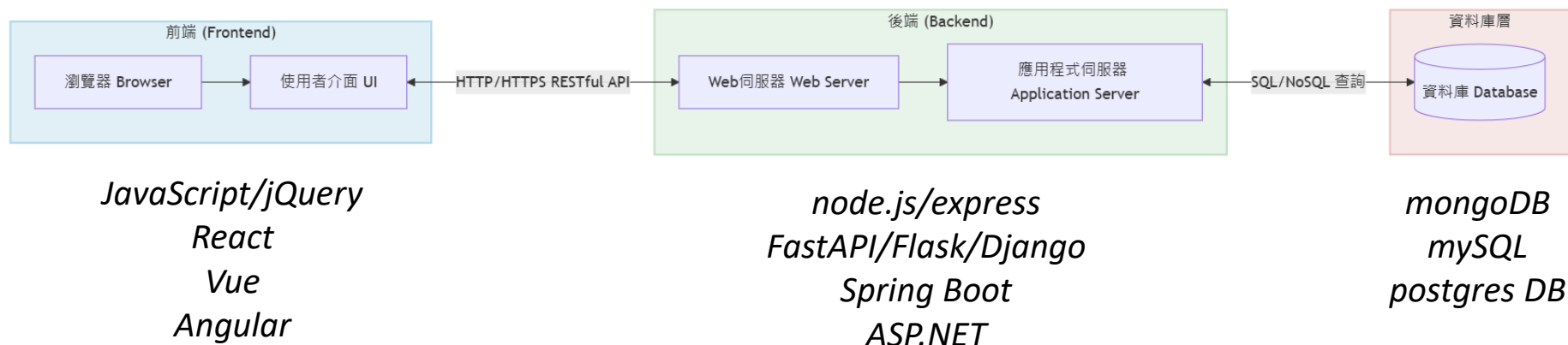
Outline

2

- Frontend vs Backend (Concept and Roles)
- Node.js Basics (Server-side JavaScript)
- Express Framework and Project Structure
- Building a Simple Web App (Calling APIs from Frontend)
- MongoDB and Mongoose (Storing and Accessing Data)
- Connecting to MongoDB Atlas and Deploying the App

Frontend and Backend

3



在程式碼層級可區分為前端與後端，在部署上前後端可以完全分離 (須設定好跨網域存取)，或是屬於同一個專案底下。

Learning Backend Skills

4

□ JavaScript

- Node.js: <https://www.w3schools.com/nodejs/>
- Express: <https://expressjs.com/>
- Deno: <https://deno.com/>

□ Python

- FastAPI: <https://fastapi.tiangolo.com/>
- Flask: <https://www.tutorialspoint.com/flask/index.htm>
- Django: <https://www.djangoproject.com/>

□ Java

- Spring Boot: <https://spring.io/projects/spring-boot>

□ PHP (Hypertext Preprocessor)

- <https://www.w3schools.com/pHP/default.asp>
- Laravel: <https://laravel.com/>

□ ASP.NET

- https://www.w3schools.com/asp/webpages_intro.asp

Node.js

5

- ❑ Node.js is an open source server environment.
- ❑ Node.js allows you to run JavaScript on the server.
- ❑ <https://www.w3schools.com/nodejs/default.asp>



Node.js File

6

- What is a Node.js File?
 - ▣ Node.js files have extension ".js".
 - ▣ Node.js files contain tasks that will be executed on certain events.
 - ▣ A typical event is someone trying to access a port on the server.
 - ▣ Node.js files must be initiated on the server before having any effect.

Installation and First Example₁

7

- Download and install node.js

- ▣ <https://nodejs.org/en/>

- Create a Node.js file named "myfirst.js":

```
const http = require('http');
```

```
http.createServer(function (req, res) {  
    res.writeHead(200, { 'Content-Type': 'text/html' });  
    res.end('Hello World!');  
}).listen(8080);
```

The code tells the computer to write "Hello World!" if anyone (e.g. a web browser) tries to access your computer on port 8080.

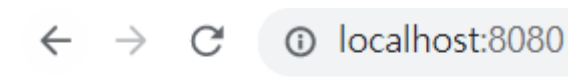
Installation and First Example₂

8

- The file you have just created must be initiated by Node.js before any action can take place.
 - ▣ Start your command line interface (CMD in Windows or “終端機” in MAC), switch to the code folder, and input the following command:

`node myfirst.js`

- ▣ Now, your computer works as a server!
 - Start your internet browser, and type in the address:
<https://localhost:8080>



Hello World!

Node.js Modules₁

9

- ❑ What is a Module in Node.js?
 - ❑ Consider modules to be the same as JavaScript libraries.
 - ❑ A set of functions you want to include in your application.
- ❑ Built-in Modules
 - ❑ Node.js has a set of built-in modules which you can use without any further installation.
 - ❑ Look at our [Built-in Modules Reference](#) for a complete list of modules.

Node.js Modules₂

10

□ Include Modules

- ▣ To include a module, use the `require()` function with the name of the module:

```
const http = require('http');
```

- ▣ Now your application has access to the HTTP module, and is able to create a server:

```
//myFirst.js
```

```
const http = require('http');
```

```
http.createServer(function (req, res) {  
  res.writeHead(200, { 'Content-Type': 'text/html' });  
  res.end('Hello World!');  
}).listen(8080);
```

Node.js HTTP Module₁

11

- Node.js has a built-in module called HTTP, which allows Node.js to transfer data over the Hyper Text Transfer Protocol (HTTP).
- To include the HTTP module, use the `require()` method:

```
const http = require('http');
```

Node.js HTTP Module₂

12

□ Node.js as a Web Server

- ▣ The HTTP module can create an HTTP server that listens to server ports and gives a response back to the client.
- ▣ Use the `createServer()` method to create an HTTP server:

//create a server object:

```
http.createServer(function (req, res) {  
    res.write('Hello World!'); //write a response to the client  
    res.end(); //end the response  
}).listen(8080); //the server object listens on port 8080
```

The function passed into the `http.createServer()` method, will be executed when someone tries to access the computer on port 8080.

Node.js HTTP Module₃

13

- If the response from the HTTP server is supposed to be displayed as HTML, you should include an HTTP header with the correct content type:

```
//demo_http.js
http.createServer(function (req, res) {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.write('Hello World!');
  res.end();
}).listen(8080);
```

The first argument of the `res.writeHead()` method is the status code, 200 means that all is OK, the second argument is an object containing the response headers (代表輸出型態是HTML).

<https://restfulapi.net/http-status-codes/>

Node.js HTTP Module₄

14

□ Read the Query String

- ▣ The function passed into the `http.createServer()` has a `req` argument that represents the request from the client, as an object (`http.IncomingMessage` object).
- ▣ This object has a property called `"url"` which holds the part of the **url** that comes after the domain name:

```
//demo_http_url
```

```
http.createServer(function (req, res) {  
    res.writeHead(200, { 'Content-Type': 'text/html' });  
    res.write(req.url);  
    res.end();  
}).listen(8080);
```

- ▣ Start your internet browser, and type in the address:

<https://localhost:8080/hello> or <https://localhost:8080/hi>

Node.js HTTP Module₅

15

□ Split the Query String

- ▣ There are built-in modules to easily split the query string into readable parts, such as the URL module.

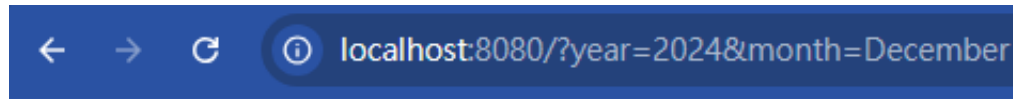
```
//demo_querystring.js
const http = require('http');
const url = require('url');

http.createServer(function (req, res) {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  let q = url.parse(req.url, true).query;
  let txt = q.year + " " + q.month;
  res.end(txt);
}).listen(8080);
```

Node.js HTTP Module₆

16

- Start your internet browser, and type in the address:
<https://localhost:8080/?year=2024&month=December>



2024 December

Node.js File System Module₁

17

- The Node.js file system module allows you to work with the file system on your computer.
- To include the File System module, use the `require()` method:

```
const fs = require('fs');
```

Node.js File System Module₂

18

Assume we have the
HTML file (demofile1.html)

```
<html><body>
  <h1>My Header</h1>
  <p>My paragraph.</p>
</body></html>
```

Create a Node.js file that reads the HTML file, and return the content:

```
const http = require('http');
const fs = require('fs');
http.createServer(function (req, res) {
  fs.readFile('demofile1.html', function (err, data) {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.write(data);
    return res.end();
  });
}).listen(8080);
```

Node.js + JSON

19

- You can easily convert JavaScript objects/arrays to JSON objects/arrays.

```
const http = require('http');  
let myObj = {name: 'Stephen Curry', team: 'GSW'};  
  
http.createServer(function (req, res) {  
  res.writeHead(200, { 'Content-Type': 'application/json' });  
  res.end(JSON.stringify(myObj));  
}).listen(8080);
```

Node.js NPM₁

20

- What is NPM?
 - ▣ NPM is a package manager (套件/函式庫管理工具) for Node.js packages (modules).
 - ▣ www.npmjs.com hosts thousands of free packages to download and use.
 - ▣ The NPM program is installed on your computer when you install Node.js.



Node.js NPM₂

21

□ What is a Package?

- ▣ A package in Node.js contains all the files you need for a module.
- ▣ Modules are JavaScript libraries you can include in your project.

□ Download a Package

- ▣ Downloading a package is very easy.
- ▣ Open the command line interface and tell NPM to download the package you want.
- ▣ If downloading a package called “upper-case”:

`npm install upper-case` (會在node_modules下建立upper-case子目錄)

Node.js NPM₃

22

- Include the "upper-case" package the same way you include any other module:

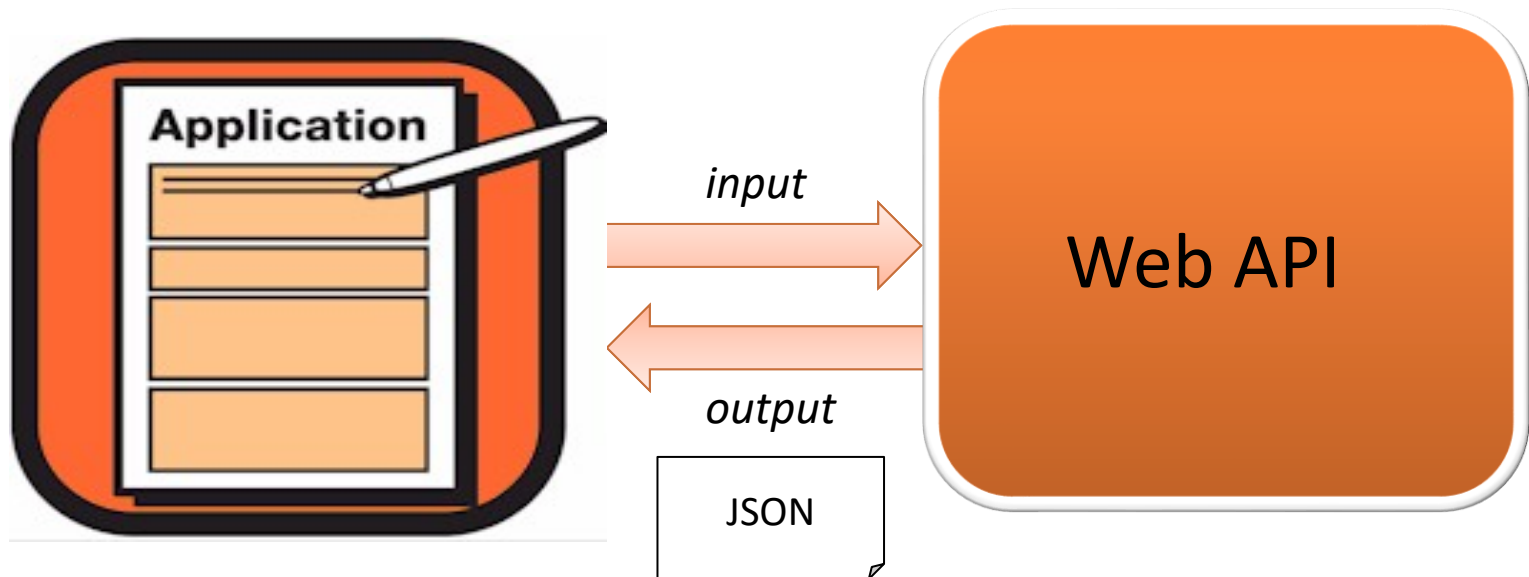
```
const http = require('http');
const uc = require('upper-case');
http.createServer(function (req, res) {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.write(uc.upperCase("Hello World!"));
  res.end();
}).listen(8080);
```

Using Node.js to Build Web APIs

Web API

24

- A Web API is a development in Web services where emphasis has been moving to simpler **REST**-based communications. (also called **RESTful service**)



HTTP Methods

(Cornerstone of RESTful Services)

25

- GET: to get the thing (resource / file) at the requested URL. (讀取)
- POST: to ask the server to accept the body info attached to the request, and give it to the thing at the requested URL. (新增)
- PUT: to put the enclosed info (the body) at the requested URL. (更換)
- Patch: to apply partial modifications to a resource. (更新)
- DELETE: to delete the thing (resource / file) at the requested URL. (刪除)

Implementing a simple API with Node.js₁

26

□ GET請求處理

- ▣ 檢查請求方法(GET/POST)和 URL
- ▣ GET 請求直接回傳資料(ratings陣列)

```
// demo_rating_api.js
const http = require('http');
const ratings = [];

http.createServer((req, res) => {
  res.setHeader('Content-Type', 'application/json');

  if (req.method === 'GET' && req.url === '/ratings') {
    return res.end(JSON.stringify(ratings));
  }
});
```

Implementing a simple API with Node.js₂

27

□ POST 請求處理

- ▣ 資料會以串流方式陸續傳送至Server
- ▣ 程式使用 'data' 事件收集資料片段
- ▣ 程式使用 'end' 事件處理完整資料
- ▣ 回應：
 - 成功：回傳ratings陣列
 - 錯誤：回傳錯誤訊息

Implementing a simple API with Node.js₃

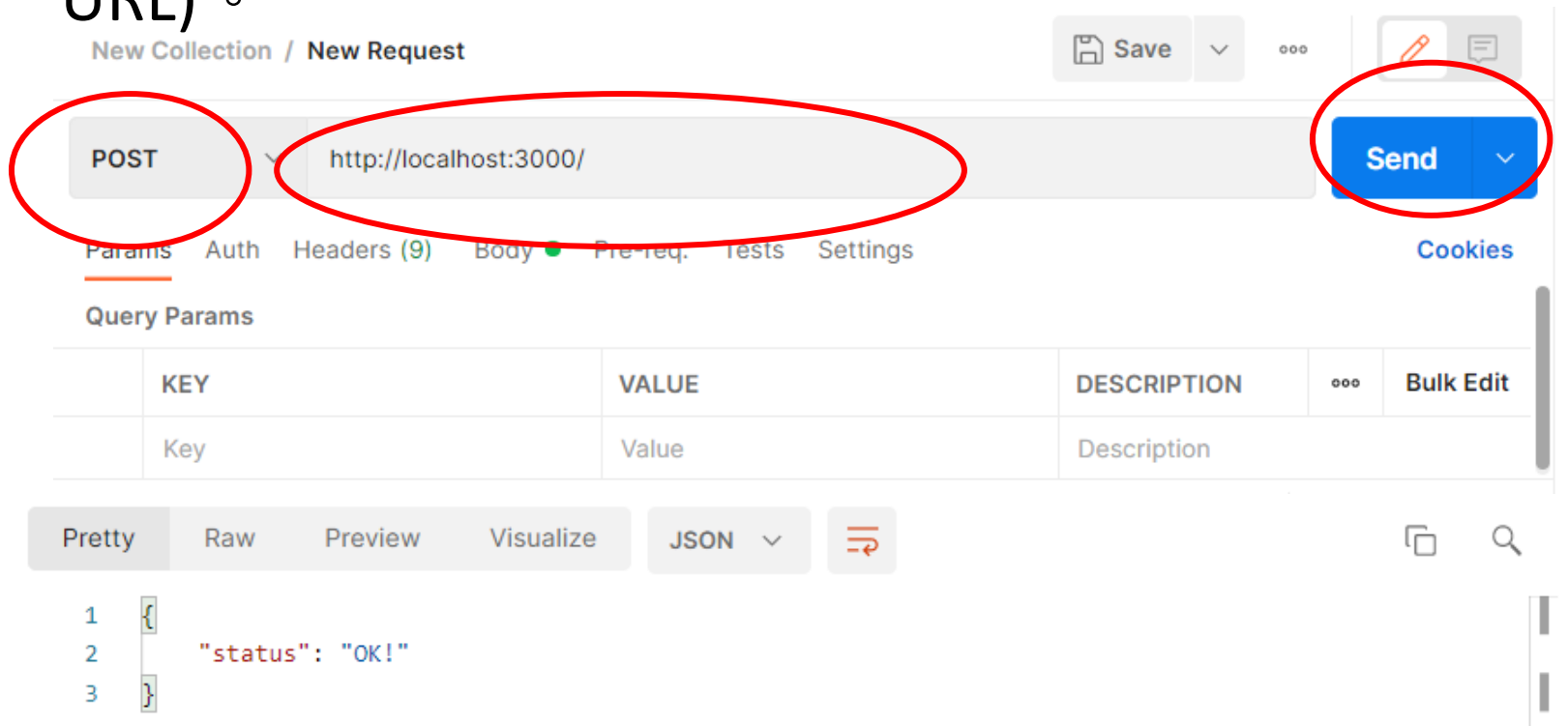
28

```
// 輸入範例{"score":5}
if (req.method === 'POST' && req.url === '/ratings') {
  let body = '';
  req.on('data', chunk => body += chunk);
  req.on('end', () => {
    const score = JSON.parse(body).score;
    if (!score || score < 1 || score > 5) {
      res.statusCode = 400;
      return res.end(JSON.stringify(
        ({ error: '評分須在 1-5 分間' })));
    }
    ratings.push({ score, time: new Date() });
    res.statusCode = 201;
    res.end(JSON.stringify(ratings));
  });
}
```

Using Postman for API Testing₁

29

- Postman: <https://www.postman.com/downloads/>
- 可使用Postman去測試以各API端點 (method + URL) 。



Using Postman for API Testing₂

30

□ 以ratings API為例：

The screenshot displays the Postman interface for an API test. At the top, the URL bar shows `http://127.0.0.1:3000/ratings` with a 'Save' icon. Below this, the request method is set to **POST** and the URL is repeated. A blue 'Send' button is on the right. The 'Body' tab is selected, showing a JSON payload: `{ "score": 4 }`. The 'Headers' tab shows 8 headers. The 'Test Results' tab shows 5 results. The response is a 201 status code, created in 5 ms, with a 253 B body. The response is displayed in the 'Pretty' view, showing a JSON array of two objects:

```
1 [
2   {
3     "score": 5,
4     "time": "2024-12-03T03:54:52.198Z"
5   },
6   {
7     "score": 4,
8     "time": "2024-12-03T03:55:00.784Z"
9   }
10 ]
```

Express: Foundation

express

32

- express 是小型 Node.js Web 應用程式架構。
- <https://expressjs.com/>
- <https://expressjs.com/zh-tw/>



Express App Generator

33

- 可使用應用程式產生器工具express，快速建立應用程式架構。

- ▣ 使用下列指令來安裝 express：

npm install express-generator -g

- ▣ 接著即可在現行工作目錄中建立一個Express 應用程式的架構。

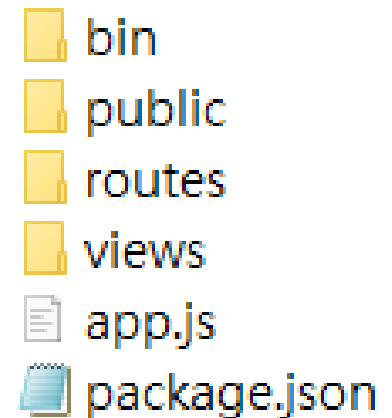
express myapp (建立一個myapp專案)

- ▣ 然後安裝相依項目：

cd myapp

npm install

- ▣ 最後即可執行
npm start



Express

Welcome to Express

Express Project Structure₁

34

□ bin:

- ▣ 專案啟動的位置，www為專案的進入點，負責啟動Node.js server，並呼叫專案底下的app.js，以啟動程式。
- ▣ 可修改www去調整port (例如從3000改為8888)。

□ node_modules:

- ▣ 專案中必要的模組以及npm安裝的模組均在此目錄。通常不會手動改這個目錄的內容。

□ public:

- ▣ 儲存靜態檔案(如HTML、CSS、前端JavaScript、圖片檔等)。
- ▣ 可以透過前端JavaScript去呼叫後端JavaScript提供的API。

Express Project Structure₂

35

□ routes:

- ▣ 存放路由文件，負責傳遞資料、設定API路由路徑等。

□ views:

- ▣ 存放顯示畫面用的樣板文件。預設格式是Jade樣板引擎。
(如是前後端完全分離架構，可先忽略此塊)

□ app.js:

- ▣ 程式進入點

□ package.json:

- ▣ 專案組態設定檔，如專案描述、自訂指令、安裝套件等。
- ▣ 預設會有一個自訂指令start。

Basic Routing₁

36

- 路由(Routing)是指判斷應用程式如何回應用戶端對特定端點(endpoint)的要求，而這個特定端點是一個 URI 與一個特定的 HTTP 要求方法(GET、POST 等)。
- 每一個路由可以有一或多個處理程式函式，當路由相符時，就會執行這些函式。
- 路由定義的結構如下：

```
app.METHOD(PATH, HANDLER)
```

- app 是 express 的實例。
- METHOD 是 HTTP 要求方法。
- PATH 是伺服器上的路徑。
- HANDLER 是當路由相符時要執行的函數。

Basic Routing₂

37

- 可修改routes子目錄底下的index.js，增加路由設定與處理函式(底下均為simple-express範例)
- ▣ 例子：對 /hello路由發出 GET 要求時的回應(JSON)：

```
router.get('/hello', function (req, res) {  
  res.send({message: 'Hello World!'});  
});
```

- ▣ 例子：對根路由(/)發出POST 要求時的回應(JSON)：

```
router.post('/', function (req,  
res) {  
  res.send({status: 'OK!'});  
});
```

Basic Routing₃

38

- ▣ 例子：對 /hello路由發出 PUT 要求時的回應(JSON)：

```
router.put('/hello', function (req, res) {  
  res.send({message: 'Hello New World!'});  
});
```

- ▣ 例子：對 /hello路由發出 DELETE 要求時的回應(JSON)：

```
router.delete('/hello', function (req, res) {  
  res.send({status: 'Done!'});  
});
```

這樣我們就建立了最為簡單(但沒有很合理)的API

Providing Static Files in Express

39

- 如果要提供影像、CSS 檔案和 JavaScript 檔案等之類的靜態檔案，請使用 Express 中的 `express.static` 內建中介軟體函數。
- ▣ 將含有靜態資料的目錄名稱傳遞給 `express.static` 中介軟體函式，就能直接開始提供檔案。
- ▣ 舉例來說，使用下列程式碼在名稱是 `public` 的目錄中，提供影像、CSS 檔案和 JavaScript 檔案：

```
app.use(express.static('public'));
```

```
app.use(express.static(path.join(__dirname, 'public')));
```

- `path.join` 是 Node.js 中的一個方法，用於組合路徑片段。
- `__dirname` 是當前執行的程式所在的目錄。

Adding a new Frontend Page

40

- 於“public”目錄下新增html檔案(如index.html)
 - ▣ CSS連結至stylesheets子目錄下的style.css
 - ▣ JavaScript連結至javascripts子目錄下的.js檔 (需新增)

Frontend Page: HTML₁

41

- 引入CSS樣式檔、jQuery函式庫(optional)、你的JavaScript程式連結。

```
<head>  
  <link rel=stylesheet type="text/css" href="stylesheets/style.css">  
  </script>  
  <script type="text/javascript" src="javascripts/myapp.js"></script>  
  <title>TodoList</title>  
</head>
```

Frontend Page: HTML₂

42

- 加入HTML主體內容。以此範例而言，是四個 button (點擊後會呼叫四個不同的function)，並增加一個div區塊用以顯示資訊。

```
<body>
  <button onclick="getTest()">GET Test</button>
  <button onclick="postTest()">Post Test</button>
  <button onclick="putTest()">PUT Test</button>
  <button onclick="deleteTest()">DELETE Test</button>
  <hr>
  <div id="display"></div>
</body>
```

Frontend Page: JavaScript₁

43

- 透過fetch函式去異步呼叫(非同步呼叫)後端API，結果再呈現於頁面中(以此案例為div區塊)。

```
function getTest() {  
  fetch("hello")  
    .then(response => response.json())  
    .then(res => {  
      document.getElementById("display").innerHTML = res.message;  
    })  
    .catch(err => {  
      console.error("GET error:", err);  
    });  
}
```

Frontend Page: JavaScript₂

44

```
function postTest() {  
  fetch("/", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json"  
    },  
    body: JSON.stringify({ name: "Curry" }) // 假裝有要發布的資料  
  })  
  .then(response => response.json())  
  .then(res => {  
    document.getElementById("display").innerHTML = res.status;  
  })  
  .catch(err => {  
    console.error("POST error:", err);  
  });  
}
```

Frontend Page: JavaScript₃

45

```
function putTest() {  
  fetch("hello", {  
    method: "PUT",  
    headers: {  
      "Content-Type": "application/json"  
    },  
    body: JSON.stringify({ name: "Curry" }) // 假裝有更新的資料  
  })  
    .then(response => response.json())  
    .then(res => {  
      document.getElementById("display").innerHTML = res.message;  
    })  
    .catch(err => {  
      console.error("PUT error:", err);  
    });  
}
```

Frontend Page: JavaScript₄

46

```
function deleteTest() {  
  fetch("hello", {  
    method: "DELETE"  
  })  
    .then(response => response.json())  
    .then(res => {  
      document.getElementById("display").innerHTML = res.status;  
    })  
    .catch(err => {  
      console.error("DELETE error:", err);  
    });  
}
```

MongoDB

MongoDB

48

- ❑ One of the most popular NoSQL database is MongoDB.
 - ▣ You can download a free MongoDB database at <https://www.mongodb.com>.
 - ▣ You can also get started with a MongoDB cloud service at <https://www.mongodb.com/cloud/atlas>.
- ❑ Install MongoDB Driver
 - ▣ To download and install the official MongoDB driver, open the Command Terminal and execute the following:

```
npm install mongodb
```


MongoDB

49

□ Creating a Database

- ▣ To create a database in MongoDB, start by creating a MongoClient object, then specify a connection URL with the correct ip address (如果是本地端就是localhost) and the name of the database you want to create.
- ▣ MongoDB will create the database if it does not exist, and make a connection to it.

MongoDB Compass

50

MongoDB Compass Community - localhost:27017/mydb

Connect View Help

Local

5 DBS 4 COLLECTIONS

☆ FAVORITE

HOST
localhost:27017

CLUSTER
Standalone

EDITION
MongoDB 4.4.5 Community

Filter your data

- > admin
- > config
- > demo
- > local
- > mydb (+) (-)
- customers
- > todo

Collections

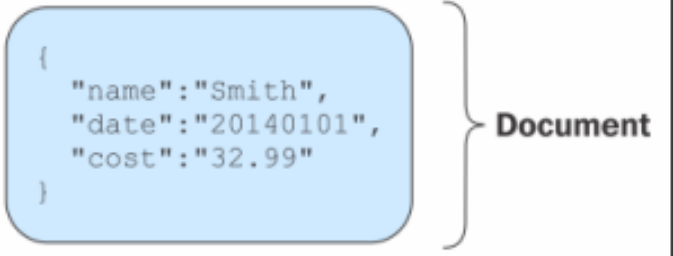
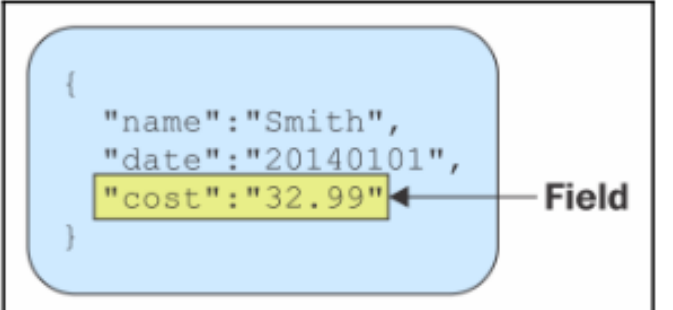
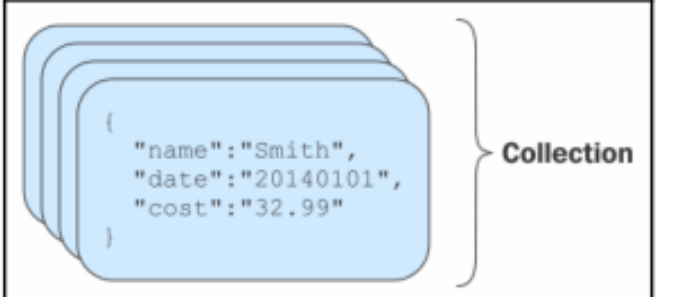
CREATE COLLECTION

Collection Name ^	Documents	Avg. Document Size	Total Document Size	Num. Indexes	Total Index Size	Properties
customers	0	-	0.0 B	1	4.0 KB	(trash icon)

此為本地端的MongoDB管理工具，可以在上面創建Database

MongoDB and RDBMS

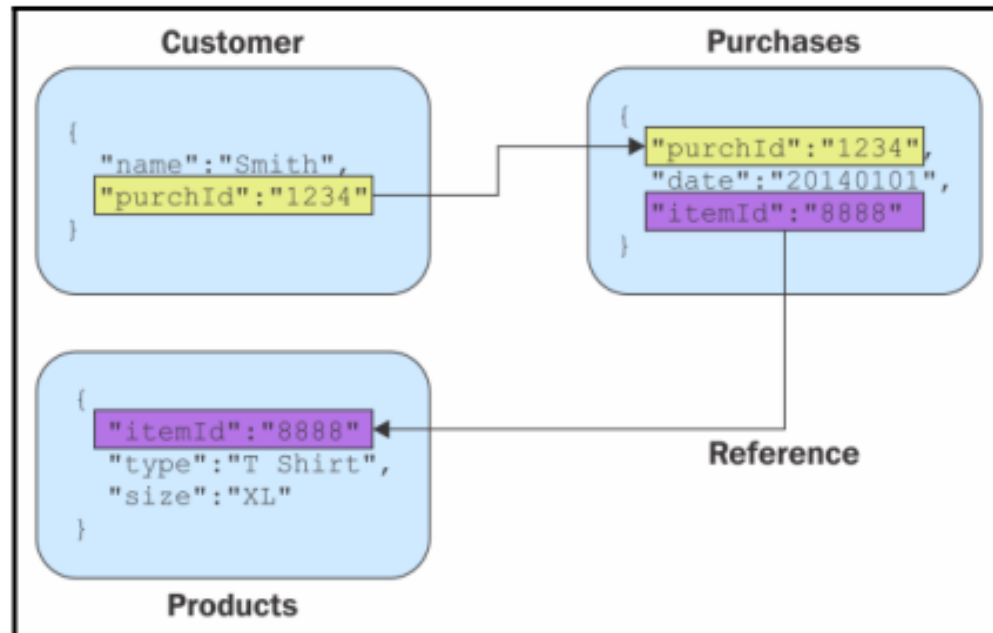
51

MongoDB	RDBMS	Illustration	
Document	Row		
Field	Column		
Collection	Table		

References

52

- Using references in MongoDB, it is possible to create a series of related collections in order to establish a normalized data model
- ▣ <https://docs.mongodb.com/manual/reference/database-references/>



Embedded Documents

53

- A data-modeling solution in MongoDB would be simply to collapse the normalized relationships and fold the related information into embedded documents.
- ▣ <https://docs.mongodb.com/manual/core/data-model-design/#data-modeling-embedding>



MongoDB Collection Example (in Compass)

54

The screenshot shows the MongoDB Compass interface for the 'mydb.customers' collection. The top bar indicates 27 documents, a total size of 1.7KB, and an average size of 64B. There is 1 index with a total size of 32.0KB and an average size of 32.0KB. The 'Documents' tab is selected, showing a list of documents. The first document is expanded, showing its fields: '_id', 'name', and 'address'.

mydb.customers Documents

mydb.customers

DOCUMENTS 27 TOTAL SIZE 1.7KB AVG. SIZE 64B INDEXES 1 TOTAL SIZE 32.0KB AVG. SIZE 32.0KB

Documents Aggregations Explain Plan Indexes

FILTER OPTIONS FIND RESET ...

ADD DATA VIEW

Displaying documents 1 - 20 of 27 REFRESH

> `_id: ObjectId("61c12afc585d483e73143aca")`
`name: "Amy"`
`address: "Apple st 652"`

`_id: ObjectId("61c12afc585d483e73143acb")`
`name: "Ben"`
`address: "Park Lane 38"`

`_id: ObjectId("61c12afc585d483e73143acc")`
`name: "Peter"`
`address: "Lowstreet 4"`

`_id: ObjectId("61c12afc585d483e73143acd")`
`name: "Viola"`
`address: "Sideway 1633"`

Mongoose

What is Mongoose?

56

- Mongoose provides a schema-based solution to model your application data.
 - ▣ It includes built-in type casting, validation, query building, business logic hooks and more, out of the box.
 - ▣ It realizes ODM (Object Data Model).
 - ▣ <https://mongoosejs.com/>

Mongoose Example

57

- Mongoose 中文參考資料：
<https://hackmd.io/@Agry/HJu3KSZoc>
 - ▣ 範例：todo-mongoose 專案
- 前置安裝程序
 - ▣ 先使用 Express Generator 生成目錄 (*express yourapp*)
 - ▣ 安裝 mongoose
 - *npm install mongoose*
 - ▣ 啟動程式
 - *npm start*

Mongoose Example: app.js

58

```
// 引入Router (routes目錄底下的todo.js)
const todoRouter = require("./routes/todo");
// 此處的/todo代表連線到該Router的基本路徑為
// https://localhost:3000/todo
app.use("/todo", todoRouter);

module.exports = app;
```

Mongoose Example: Router₁

59

- 於routes/todo.js設定router。
- 於router中設定資料庫連結邏輯。

```
// 連接到 MongoDB 資料庫
mongoose.connect('mongodb://localhost:27017/tododb');
const db = mongoose.connection;

// 與資料庫連線發生錯誤時
db.on('error', console.error.bind(console, 'Connection
fails!'));

// 與資料庫連線成功連線時
db.once('open', function () {
  console.log('Connected to database...');
});
```

Mongoose Example: Router₂

60

- 於router中設定data model。
 - ▣ 此model對應之MongoDB Collection名稱為設定名稱的複數(此案例為todos)。

<https://mongoosejs.com/docs/models.html>

```
// 該collection的格式設定
const todoSchema = new mongoose.Schema({
  thing: { //事項名稱
    type: String, //設定該欄位的格式
    required: true //設定該欄位是否必填
  },
  isDone: { //是否已完成
    type: Boolean,
    required: true,
    default: false //設定預設值
  },
  createdAt: { //新增的時間
    type: Date,
    default: Date.now,
    required: true
  },
});

const Todo = mongoose.model('Todo', todoSchema);
```

Mongoose Example: Router₃

61

- 於routes/todo.js設定router相關邏輯。

```
// 使用非同步，才能夠等待資料庫回應
router.get("/", async (req, res) => {
  try {
    // 找出Todo資料資料表中的全部資料
    const todo = await Todo.find();
    // 將回傳的資訊轉成Json格式後回傳
    res.json(todo);
  } catch (err) {
    // 如果資料庫出現錯誤時回報 status:500 並回
    // 傳錯誤訊息
    res.status(500).json({ message:
err.message })
  }
}); https://mongoosejs.com/docs/api/model.html
```

Mongoose Example: Router₄

62

```
// 新增待辦事項，將Method改為Post
router.post("/", async (req, res) => {
  // 從req.body中取出資料
  const todo = new Todo({
    thing: req.body.thing,
    isDone: req.body.isDone,
  });
  try {
    // 使用.save()將資料存進資料庫
    const newTodo = await todo.save();
    // 回傳status:201代表新增成功 並回傳新增的資料
    res.status(201).json(newTodo);
  } catch (err) {
    // 錯誤訊息發生回傳400 代表使用者傳入錯誤的資訊
    res.status(400).json({ message: err.message });
  }
});
```

Mongoose Example: Router₅

63

```
// 在網址中傳入id用以查詢
router.get("/:id", async (req, res) => {
  try {
    const todo = await Todo.findById(req.params.id);
    if (todo == undefined) {
      return res.status(404).json({ message: "Can't find todo" });
    } else {
      return res.status(200).json(todo);
    }
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
});
```

Mongoose Example: Router₆

64

```
// 更新代辦事項
router.put("/:id", async (req, res) => {
  try {
    // 將取出的代辦事項更新
    const newTodo = await Todo.findByIdAndUpdate(req.params.id,
      req.body, { new: true });
    res.json(newTodo);
  } catch (err) {
    // 資料庫操作錯誤將回傳500及錯誤訊息
    res.status(500).json({ message: "update todo failed!" });
  }
});
```

`{ new: true }`的作用是在更新後，回傳更新後的資料內容，而不是更新之前的內容

Mongoose Example: Router₇

65

```
// 刪除代辦事項
router.delete("/:id", async (req, res) => {
  try {
    // 將取出的代辦事項刪除
    await Todo.findByIdAndDelete(req.params.id);
    // 回傳訊息
    res.json({ message: "Delete todo successfully!" });
  } catch (err) {
    // 資料庫操作錯誤將回傳500及錯誤訊息
    console.log(err);
    res.status(500).json({ message: "remove todo failed!" });
  }
});
```

Front-End Page (Partial)

66

```
// 使用 async/await 取代 jQuery
document.addEventListener('DOMContentLoaded', async function () {
// 取得待辦事項列表
  try {
    const response = await fetch('todo');
    const todos = await response.json();
    const todoList = document.getElementById('todo-list');
    todos.forEach(function (todo) {
      const row = `|  |
| --- |
| ${new Date(todo.createdAt).toLocaleString()}</td><td>${todo.thing}</td> <td>${todo.isDone ? "完成" : "未完成"}</td></tr>`;       todoList.insertAdjacentHTML('beforeend', row);     });   } catch (error) {     console.error('取得待辦事項失敗:', error);   } } |

```

MongoDB Atlas

MongoDB Atlas

68

- MongoDB Atlas is a cloud MongoDB.
 - ▣ <https://www.mongodb.com/cloud/atlas>
 - ▣ Atlas handles deploying, managing, and healing your deployments on the cloud (AWS, Azure, and GCP).
 - ▣ Avoiding the deployment issue of DB.

Configuring Security in Atlas₁

69

- Database Access: create a user to be the database administrator. (設定可存取雲端DB之帳密)

DATA STORAGE

Clusters

Triggers

Data Lake BETA

SECURITY

Database Access

Network Access

Advanced

ORGANIZATION 1828312 > PROJECT 0

Database Access

Database Users Custom Roles

+ ADD NEW DATABASE USER

User Name ↕	Authentication Method ▲	MongoDB Roles	Actions
👤 admin	SCRAM	readWriteAnyDatabase@admin	<button>EDIT</button> <button>DELETE</button>

Configuring Security in Atlas₂

70

- Atlas only allows client connections to a cluster from entries in the project's whitelist.
- You can set the IP to 0.0.0.0/0 to allow access from anywhere. (若設定固定IP更為安全)
 - ▣ <https://docs.atlas.mongodb.com/security-whitelist/>

DATA STORAGE

Clusters

Triggers

Data Lake BETA

SECURITY

Database Access

Network Access

Advanced

ORGANIZATION 1828312 > PROJECT 0

Network Access

IP Whitelist

Peering

Private Endpoint

+ ADD IP ADDRESS

You will only be able to connect to your cluster from the following list of IP Addresses:

IP Address	Comment	Status	Actions
0.0.0.0/0 (includes your current IP address)		● Active	<button>EDIT</button> <button>DELETE</button>

Creating Database

71

Atlas

Realm

Charts

OverviewReal TimeMetricsCollectionsSearchProfilerPerformance AdvisorOnline ArchiveCommand Line Tools

DATABASES: 0COLLECTIONS: 0

VISUALIZE YOUR DATAREFRESH



Explore Your Data

- **Find:** run queries and interact with documents
- **Indexes:** build and manage indexes
- **Aggregation:** test aggregation pipelines
- **Search:** build search indexes

Load a Sample Dataset

Add My Own Data

[Learn more in Docs and Tutorials](#)

Create Database

Database name ?

mydb

Collection name ?

customers

Additional Preferences

☐ Capped Collection ?

Cancel

Create

Connecting Node.js to Atlas₁

72

Connect to Cluster0

✓ Setup connection security

Choose a connection method

Connect

Choose a connection method [View documentation](#) 

Get your pre-formatted connection string by selecting your tool below.



Connect with the MongoDB Shell

Interact with your cluster using MongoDB's interactive Javascript interface



Connect your application

Connect your application to your cluster using MongoDB's native drivers



Connect using MongoDB Compass

Explore, modify, and visualize your data with MongoDB's GUI



Connecting Node.js to Atlas₂

73

- Copy the connection string for Node.js

×

Connect to Cluster0

✓ Setup connection security > ✓ Choose a connection method > Connect

1 Select your driver and version

DRIVER

Node.js

VERSION

4.0 or later

2 Add your connection string into your application code

☐ Include full driver code example

```
mongodb+srv://admin:<password>@cluster0.ctev7.mongodb.net/myFirstDatabase?
retryWrites=true&w=majority
```

Replace **<password>** with the password for the **admin** user. Replace **myFirstDatabase** with the name of the database that connections will use by default. Ensure any option params are [URL encoded](#).

Connecting Node.js to Atlas₃

74

- We can simply change the url of database to the connection string in the previous slide, we can connect Mongo Atlas in the cloud, rather than the local MongoDB.
 - ▣ Example of connection string: such as
“*mongodb+srv://admin:<password>@cluster0.gt34a.mongodb.net/myFirstDatabase?retryWrites=true&w=majority*”.
 - ▣ 若以先前mongoose之範例，則資料庫的url改為上述之connection string即可。

透過Render部署Express應用程式到雲端

75

- 可透過render.com進行應用程式部署
 - ▣ 右上方選擇 Web Service，並連結GitHub/GitLab儲存庫 (Git Provider)。
 - ▣ 選擇執行環境(如Node.js)。
 - ▣ 設定建構指令(如 npm install)與啟動命令(如 npm start)。
 - ▣ 點擊 Create Web Service，Render 會自動啟動伺服器。
- 其他render教學文章
 - ▣ <https://medium.com/starbugs/render-來試試用來取代-heroku-的服務吧-render-的網路服務部署介紹-b728e86d5716>
 - ▣ <https://ithelp.ithome.com.tw/articles/10255630>

Any Question?

76

